

KNTU  
Faculty of Mathematics and Computer  
Science

## A time series project

---

**Name:** Koorosh ASil Gharehbaghi

**Student ID:** 40124463

**Instructor:** Dr. Malek Zadeh

**Project:** ARIMA(1,1) — Analysis, Training,  
Forecasting

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
import numpy as np
import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: df = pd.DataFrame(pd.read_csv("./data/daily-total-female-births-CA.csv"))
df["date"] = pd.to_datetime(df["date"])
print(df.info())
ts = df["births"]
df.head()
```

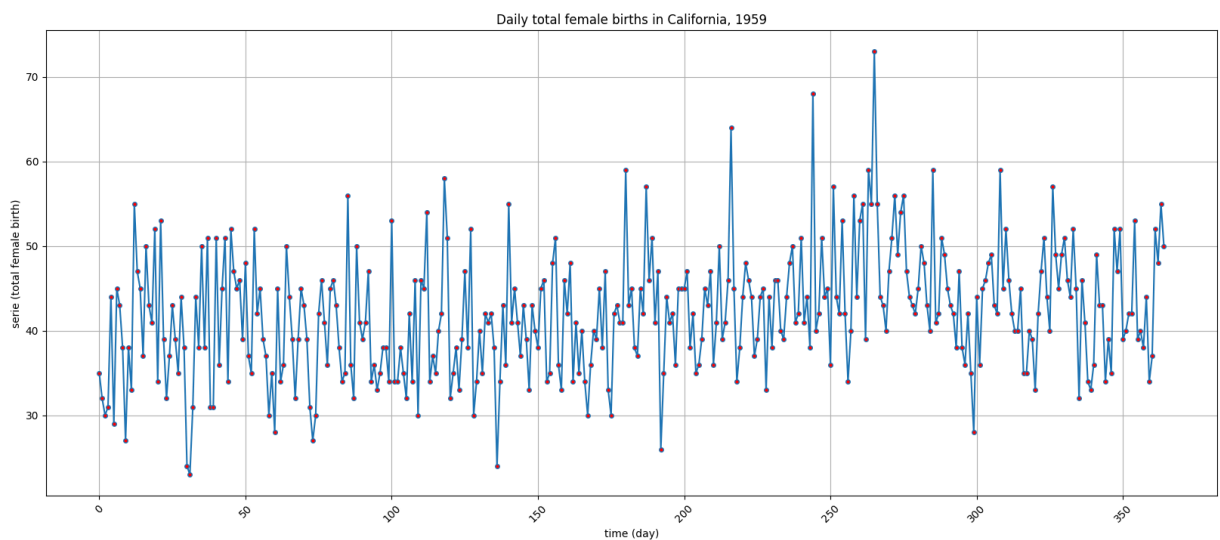
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 365 entries, 0 to 364
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   date    365 non-null     datetime64[ns]
 1   births  365 non-null     int64   
dtypes: datetime64[ns](1), int64(1)
memory usage: 5.8 KB
None
```

```
Out[2]:
```

|   | date       | births |
|---|------------|--------|
| 0 | 1959-01-01 | 35     |
| 1 | 1959-01-02 | 32     |
| 2 | 1959-01-03 | 30     |
| 3 | 1959-01-04 | 31     |
| 4 | 1959-01-05 | 44     |

## Plotting our data

```
In [3]: plt.figure(figsize=(20,8))
plt.plot(df.index, ts, markerfacecolor= "r", marker = "o", markersize = 4 )
plt.grid (True)
plt.title('Daily total female births in California, 1959')
plt.xlabel("time (day)")
plt.ylabel("serie (total female birth)")
plt.xticks(rotation = 45)
plt.show()
```



### Step 1 — Basic Descriptive Statistics

Before doing anything with stationarity, AR, MA, or ARMA, we need to understand the raw series. This step gives us a **numerical first look** at the time series:

- The **mean** tells us the average number of daily births.
- The **standard deviation** and **variance** show how much the values fluctuate.
- The **min** and **max** help detect unusual spikes.

These values help us recognize whether the series might be stable or drifting over time.

This step does **not** decide stationarity — it simply prepares our intuition for the next steps.

```
In [4]: # ts = df['births']
print("TimeSeries statistics: ")
ts.describe()
```

TimeSeries statistics:

```
Out[4]: count    365.000000
mean      41.980822
std       7.348257
min       23.000000
25%      37.000000
50%      42.000000
75%      46.000000
max       73.000000
Name: births, dtype: float64
```

## Step 2 — Compare the First Half and Second Half of the Time Series

Weak stationarity means that the **mean** and **variance** do not change much over time.

To check this in a simple way:

- We split the series into two equal halves.
- We compute the **mean** and **variance** of each half.

If the two halves have **similar mean and variance**, then the process is likely stable.

If they differ significantly, it may indicate a trend or changing volatility.

This step does **not** give a final decision — it simply guides us toward whether we might need to difference the data later.

```
In [5]: mid = len(ts) // 2
first = ts.iloc[:mid]
second = ts.iloc[mid:]
mean_first = first.mean()
var_first = first.var()
mean_second = second.mean()
```

```

var_second = second.var()
mean_diff = mean_second - mean_first
var_diff = var_second - var_first
mean_diff_pct = (mean_diff / mean_first) * 100
var_diff_pct = (var_diff / var_first) * 100
print(f"total length: {len(ts)}, split index: {mid}, first half rows: {len(first)},
print(f"mean_first: {mean_first:.4f}")
print(f"var_first: {var_first:.4f}")
print(f"mean_second: {mean_second:.4f}")
print(f"var_second: {var_second:.4f}")
print(f"absolute difference in means: {mean_diff:.4f}")
print(f"percent difference in means: {mean_diff_pct:.2f}%")
print(f"absolute difference in variances: {var_diff:.4f}")
print(f"percent difference in variances: {var_diff_pct:.2f}%")

```

```

total length: 365, split index: 182, first half rows: 182, second half rows: 183
mean_first: 39.7637
var_first: 49.4853
mean_second: 44.1858
var_second: 48.9763
absolute difference in means: 4.4221
percent difference in means: 11.12%
absolute difference in variances: -0.5090
percent difference in variances: -1.03%

```

## Report 2 — Half-to-Half Stationarity Check (Summary)

We compared the first and second halves of the series to see if mean and variance remain stable.

### Results

- **Mean:** 39.76 → 44.19 (+11.1%)
- **Variance:** 49.49 → 48.98 (−1.0%)

### Interpretation

- The **mean increases noticeably** (about 11%). This hints at a mild upward drift → possible violation of weak stationarity.
- The **variance is stable**, suggesting no change in volatility.

### Conclusion

The series is *almost* stationary but may have a small trend in the mean. We need formal tests (ADF + KPSS) to confirm whether differencing is required.

## Step 3 — Purpose of ADF and KPSS Tests

Now that we inspected the mean and variance manually, we use formal statistical tests to check whether the series is weakly stationary.

We use two standard tests:

### ADF (Augmented Dickey–Fuller)

- **Null hypothesis:** The series is non-stationary.
- If  $p\text{-value} < 0.05$ , we reject the null  $\rightarrow$  suggests **stationary**.

### KPSS (Kwiatkowski–Phillips–Schmidt–Shin)

- **Null hypothesis:** The series is stationary.
- If  $p\text{-value} < 0.05$ , we reject the null  $\rightarrow$  suggests **non-stationary**.

Using both is the standard approach:

- **ADF detects unit roots** (non-stationarity).
- **KPSS detects level-stationarity**.
- *Agreement between them* leads to a clear conclusion.

This step tells us whether differencing is required before fitting AR/MA/ARMA models.

```
In [6]: adf_result = adfuller(ts)
print("ADF statistic:", adf_result[0])
print("ADF p-value:", adf_result[1])

kpss_result = kpss(ts, regression='c', nlags='auto')
print("KPSS statistic:", kpss_result[0])
print("KPSS p-value:", kpss_result[1])
```

ADF statistic: -4.808291253559763  
ADF p-value: 5.243412990149865e-05  
KPSS statistic: 1.6129657593856863  
KPSS p-value: 0.01

## Report 3 — Interpretation of ADF & KPSS

### 1. Results

| Test | Statistic | p-value  | Interpretation                                                |
|------|-----------|----------|---------------------------------------------------------------|
| ADF  | -4.808    | 0.000052 | Reject $H_0 \rightarrow$ <i>series appears stationary</i>     |
| KPSS | 1.613     | 0.01     | Reject $H_0 \rightarrow$ <i>series appears non-stationary</i> |

### 2. Why do they contradict?

ADF  $\rightarrow$  stationary

KPSS  $\rightarrow$  non-stationary

This usually means the series is:

- *Trend-stationary* (stationary after removing a deterministic trend)
- Affected by *structural breaks*
- Exhibiting *seasonality*
- Having *shifting mean levels* across the year

These patterns can pass one test and fail the other. A statistic is a number calculated from your time series that measures how strongly the data supports or contradicts the null hypothesis.

### 3. Practical Takeaway

Because the tests disagree, the series is likely *not fully stationary*.

Next steps usually are:

- Inspect trend
- Inspect seasonality
- Apply differencing or seasonal differencing

- Re-run stationarity tests after differencing

## Step 4

### What are we doing and why?

The ADF and KPSS tests gave **conflicting results**, which is very common with real time-series.

To resolve this, the typical next step is:

1. **Visually inspect the differenced series:** Differencing removes trend or slow drifts.
2. **Check if the differenced series looks stationary:** A stationary series will look like random noise with no visible upward or downward drift.
3. **Re-run ADF and KPSS on the differenced series:** If both tests now agree (ADF: stationary, KPSS: stationary), then the original series is integrated of order  $1 \rightarrow I(1)$ .

This step helps us decide:

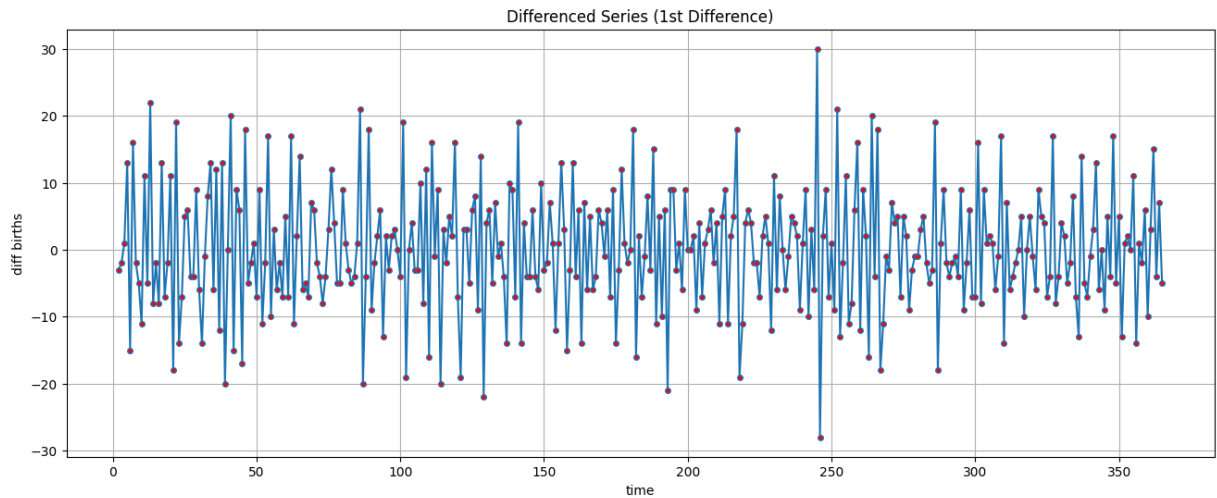
- Whether ARMA is enough, or
- Whether we must use ARIMA with  $d = 1$ .

```
In [14]: ts_diff = ts.diff().dropna()

plt.figure(figsize=(16,6))
plt.plot(ts_diff.index, ts_diff, markerfacecolor= "r", marker = "o", markersize = 4)
plt.grid(True)
plt.title("Differenced Series (1st Difference)")
plt.xlabel("time")
plt.ylabel("diff births")
plt.show()

adf_stat, adf_p, _, _, _ = adfuller(ts_diff)
kpss_stat, kpss_p, _, _ = kpss(ts_diff, regression='c')

print(f"ADF statistic (diff): {adf_stat}")
print(f"ADF p-value (diff): {adf_p}\n")
print(f"KPSS statistic (diff): {kpss_stat}")
print(f"KPSS p-value (diff): {kpss_p}")
```



ADF statistic (diff): -12.625178633871343  
ADF p-value (diff): 1.5472400091969784e-23

KPSS statistic (diff): 0.18738808325724707  
KPSS p-value (diff): 0.1

## Report 4 — Interpretation of Differenced Series

- The plot of the differenced series shows values fluctuating around zero with roughly constant variance.
- **ADF test:** statistic = -12.625, p-value  $\approx 0 \rightarrow$  strongly reject non-stationarity  $\rightarrow$  series is stationary.
- **KPSS test:** statistic = 0.187, p-value = 0.1  $\rightarrow$  fail to reject stationarity  $\rightarrow$  series is stationary.
- Both tests now agree that the differenced series is stationary.
- Conclusion: the original series is integrated of order 1 ( $I(1)$ ), meaning it becomes stationary after one differencing.
- Next step: examine **ACF** and **PACF** to determine appropriate AR and MA orders for ARIMA modeling.

## Step 5 — Examine ACF and PACF

We now analyze the autocorrelation and partial autocorrelation of the differenced series:

- **ACF (Autocorrelation Function):** shows correlation between current value and its past values (lags).

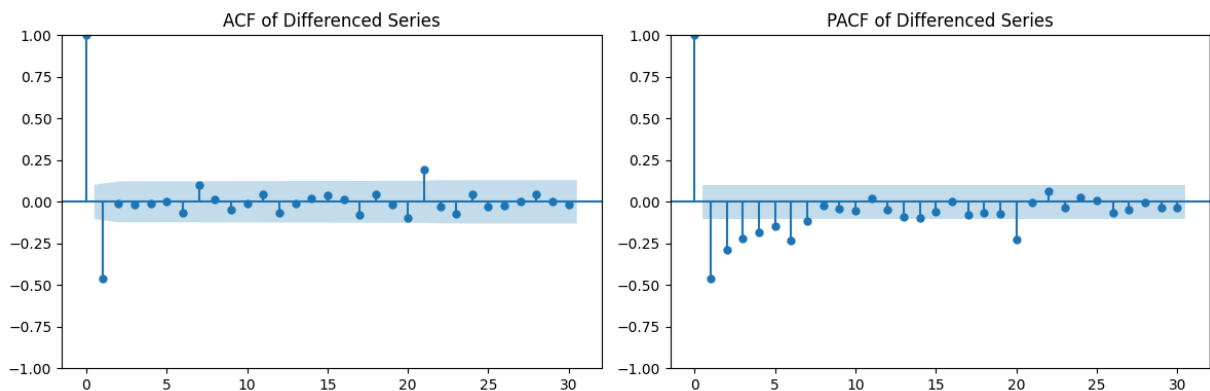
- **PACF (Partial Autocorrelation Function):** shows correlation with past values after removing effects of intermediate lags.
- These plots help us decide the **p (AR order)** and **q (MA order)** for  $\text{ARIMA}(p,1,q)$  modeling.
- Only the differenced series (stationary) should be analyzed with ACF/PACF.

```
In [8]: fig, axes = plt.subplots(1, 2, figsize=(12,4))

plot_acf(ts_diff, lags=30, ax=axes[0])
axes[0].set_title("ACF of Differenced Series")

plot_pacf(ts_diff, lags=30, method='yw', ax=axes[1])
axes[1].set_title("PACF of Differenced Series")

plt.tight_layout()
plt.show()
```



## Report 5 — Interpreting ACF & PACF of Differenced Series

### ACF (top plot):

- Only the first lag has a noticeable spike outside the confidence bounds.
- All other lags are within the shaded region → weak autocorrelation beyond lag 1.
- Suggests a **MA(1)** component might be sufficient.

### Overall takeaway:

- Differenced series looks stationary (consistent with Step 4 tests).
- For  $\text{MA}(q)$ , the plots suggest starting with **MA(1)** as a candidate model.
- These are starting points; later we can refine using AIC/BIC or cross-validation.

---

## Step 6 — Fit ARIMA Model & Forecast

Now that the series is stationary and we have an idea of AR ( $p$ ) and MA ( $q$ ) orders from Step 5:

- Fit an **MA(1,q)** model to the original series.
- $p$  from PACF,  $d=1$  because we differenced once,  $q$  from ACF.
- Check model diagnostics (residuals should look like white noise).

---

```
In [13]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.arima_process import arma_generate_sample

p, d, q = 0, 1, 1

ts = df['births'].reset_index(drop=True).astype(float)
ts.index = ts.index + 1

model = ARIMA(ts, order=(p, d, q))
model_fit = model.fit()

ma = model_fit.params['ma.L1']
sigma2 = model_fit.params['sigma2']

ar_coefs = np.array([1])
ma_coefs = np.array([1, ma])

print(model_fit.summary())

steps = 70
np.random.seed(42)

sim_diffs = arma_generate_sample(ar=ar_coefs, ma=ma_coefs, nsample=steps, scale=np.

start_idx = ts.index.max() + 1
sim_index = range(start_idx, start_idx + steps)

if d == 1:
    last_value = ts.iloc[-1]
    sim_levels = last_value + np.cumsum(sim_diffs)
    sim_series = pd.Series(sim_levels, index=sim_index)
else:
    sim_series = pd.Series(sim_diffs, index=sim_index)

forecast = model_fit.get_forecast(steps=steps)
```

```

mean = forecast.predicted_mean
ci = forecast.conf_int()

plt.figure(figsize=(20, 8))
plt.plot(ts.index, ts.values, label='Observed', color='steelblue', linewidth=2, mar
plt.plot(sim_series.index, sim_series.values, label='Simulated MA(1)', color='crims
plt.plot(mean.index, mean.values, label=f'Forecast {steps} steps', color='forestgre
plt.fill_between(ci.index, ci.iloc[:, 0], ci.iloc[:, 1], color='forestgreen', alpha
plt.xlabel("Day")
plt.ylabel("births")
plt.title("Observed + MA(1) Simulation + MA(1) Forecast")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

```

### SARIMAX Results

```

=====
Dep. Variable:          births    No. Observations:          365
Model:                 ARIMA(0, 1, 1)    Log Likelihood          -1229.110
Date:                  Wed, 04 Feb 2026    AIC                    2462.221
Time:                  18:51:01    BIC                    2470.015
Sample:                0    HQIC                    2465.319
                        - 365
Covariance Type:       opg
=====

```

|        | coef    | std err | z       | P> z  | [0.025 | 0.975] |
|--------|---------|---------|---------|-------|--------|--------|
| ma.L1  | -0.9480 | 0.018   | -52.898 | 0.000 | -0.983 | -0.913 |
| sigma2 | 49.8582 | 3.310   | 15.063  | 0.000 | 43.371 | 56.346 |

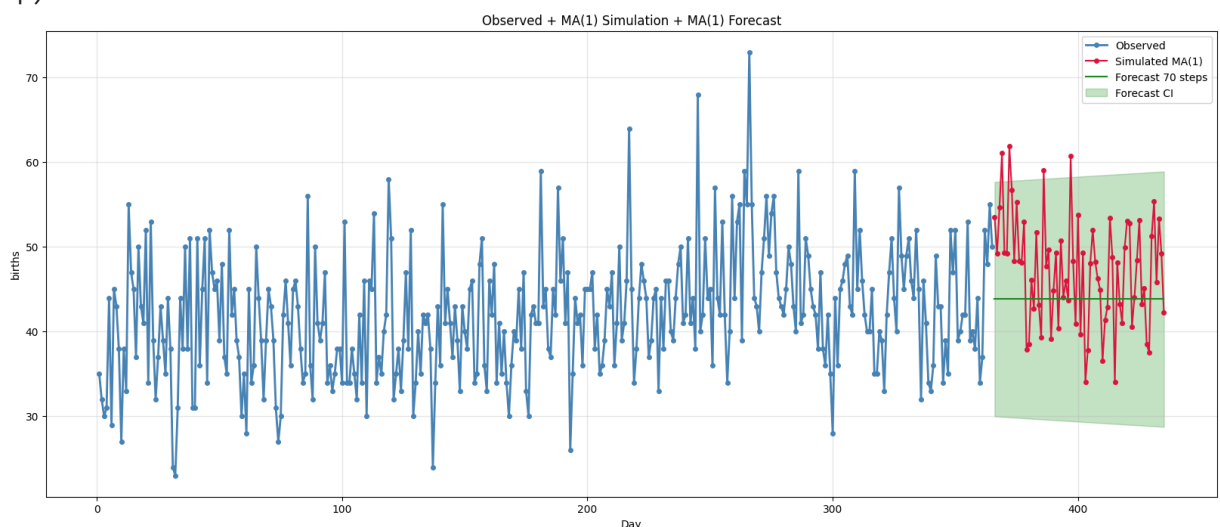
```

=====
Ljung-Box (L1) (Q):          4.22    Jarque-Bera (JB):          20.22
Prob(Q):                    0.04    Prob(JB):              0.00
Heteroskedasticity (H):      0.97    Skew:                  0.51
Prob(H) (two-sided):        0.88    Kurtosis:              3.54
=====

```

### Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).



## Report 6 — Short Version (updated)

**Model fitted:** ARIMA(0, 1, 1) (an MA(1) on first differences)

- **MA(1) coefficient:**  $\hat{\theta}_1 = -0.9480$  (std. err. = 0.018,  $p < 0.001$ )
- **Innovation variance:**  $\hat{\sigma}^2 = 49.8582$  (std. err. = 3.310)
- **Sample size:**  $N = 365$
- **Log Likelihood:**  $-1229.110$ , **AIC:** 2462.221, **BIC:** 2470.015
- **Ljung–Box (L1):**  $Q = 4.22$ ,  $\text{Prob}(Q) = 0.04$  — indicates residual autocorrelation at lag 1
- **Jarque–Bera:** JB  $p = 0.00$  — residuals deviate from normality

## Final mathematical formulas (rendered)

Define the first difference  $\Delta X_t = X_t - X_{t-1}$ .

The fitted difference equation is:

$$\Delta X_t = \varepsilon_t + \theta_1 \varepsilon_{t-1}, \quad \hat{\theta}_1 = -0.9480$$

Equivalently, in levels:

$$X_t = X_{t-1} + \varepsilon_t - 0.9480 \varepsilon_{t-1}$$

The innovations are estimated as:

$$\varepsilon_t \sim \mathcal{N}(0, 49.8582), \quad \hat{\sigma} \approx 7.062$$